ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<2>>

Descripción

```python
def req_2(catalog, pais, min_cajas, max_cajas):
    """
    Retorna el resultado del requerimiento 2
    """
    start_time = get_time()

    pedidos_pais = msc.get(catalog["OrdenPaisSC"], pais)
    pedidos_filtrados = lt.new_list()
    frecuencias = msc.new_map(100, 1)

    total_pedidos = 0
    productos_diferentes = 0
    suma_marketing = 0
    suma_amount = 0
    producto_frecuente = None
    frecuencia_maxima = 0

    if pedidos_pais is not None:
        for i in range(lt.size(pedidos_pais)):
            pedido = lt.get_element(pedidos_pais, i)
            cajas = pedido["Boxes_Shipped"]

            if min_cajas <= cajas <= max_cajas:
                lt.add_last(pedidos_filtrados, pedido)
                total_pedidos += 1

                suma_marketing += pedido["Marketing_Spend"]
                suma_amount += pedido["Amount"]

                producto = pedido["Product"]
                cantidad = msc.get(frecuencias, producto)

                if cantidad is None:
                    cantidad = 1
                    productos_diferentes += 1
                else:
                    cantidad += 1

                msc.put(frecuencias, producto, cantidad)

                if cantidad > frecuencia_maxima:
                    frecuencia_maxima = cantidad
                    producto_frecuente = producto

                elif cantidad == frecuencia_maxima:
                    if producto_frecuente is None or producto < producto_frecuente:
                        producto_frecuente = producto

    promedio_marketing = 0
    promedio_amount = 0

    if total_pedidos > 0:
        promedio_marketing = suma_marketing / total_pedidos
        promedio_amount = suma_amount / total_pedidos
```

```python
def comparar(pedido1, pedido2):
    cajas1 = pedido1["Boxes_Shipped"]
    cajas2 = pedido2["Boxes_Shipped"]

    if cajas1 != cajas2:
        return cajas1 < cajas2

    fecha1 = pedido1["Order_Date"]
    fecha2 = pedido2["Order_Date"]

    if fecha1 != fecha2:
        return fecha1 > fecha2

    return pedido1["Order_ID"] <= pedido2["Order_ID"]

lt.merge_sort(pedidos_filtrados, comparar)

pedidos_mostrar = lt.new_list()

if total_pedidos > 20:
    for i in range(10):
        pedido = lt.get_element(pedidos_filtrados, i)
        lt.add_last(pedidos_mostrar, pedido)

    for i in range(total_pedidos - 10, total_pedidos):
        pedido = lt.get_element(pedidos_filtrados, i)
        lt.add_last(pedidos_mostrar, pedido)
else:
    pedidos_mostrar = pedidos_filtrados

end_time = get_time()
tiempo = delta_time(start_time, end_time)

return {
    "tiempo": tiempo,
    "total_pedidos": total_pedidos,
    "productos_diferentes": productos_diferentes,
    "promedio_marketing": promedio_marketing,
    "promedio_amount": promedio_amount,
    "producto_frecuente": producto_frecuente,
    "frecuencia_maxima": frecuencia_maxima,
    "pedidos": pedidos_mostrar
}
```

Consulta los pedidos de un país cuyo número de cajas enviadas (Boxes_Shipped) se encuentre dentro de un rango inclusivo. Durante la carga se construye la tabla hash OrdenPaisSC (Separate Chaining), que utiliza el país como llave y almacena un array_list con sus pedidos, permitiendo acceder a ellos sin recorrer toda la lista principal.

La consulta filtra los registros y calcula la cantidad de pedidos, el número de productos distintos, los promedios de inversión en mercadeo y monto, y el producto más frecuente. Para contar las frecuencias se utiliza otra tabla hash de Separate Chaining, donde la llave es el nombre del producto y el valor es su cantidad de apariciones.

Los pedidos filtrados se ordenan con Merge Sort por cajas ascendentes, fecha descendente y Order_ID ascendente. Si se obtienen más de 20 pedidos, se presentan los primeros diez y los últimos diez.

Entrada	catalog, pais, min_cajas, max_cajas
Salidas	tiempo, total_pedidos, productos_diferentes, promedio_marketing, promedio_amount, producto_frecuente, frecuencia_maxima, pedidos.
Implementado (Sí/No)	Implementado por Katy Teran

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Buscar el país en OrdenPaisSC mediante msc.get (Separate Chaining)	O(1) promedio
Recorrer los p pedidos del país y filtrar por rango de cajas	O(p)
Consultar y actualizar frecuencias de productos en una tabla hash	O(1) promedio por pedido
Calcular los promedios con los acumuladores	O(1)
Ordenar los k pedidos que cumplen con Merge Sort	O(k log k)
Construir la lista de presentación (hasta 20 pedidos)	O(1)
TOTAL	***O(p + k log k)***

Análisis

El uso de OrdenPaisSC me permite reducir la cantidad de pedidos que debo revisar, ya que la búsqueda en la tabla hash tiene un costo O(1) promedio y solo necesito recorrer los p pedidos del país seleccionado. Durante ese recorrido también actualizo las frecuencias de los productos mediante otra tabla hash, evitando hacer búsquedas adicionales.

El mayor costo puede estar en el ordenamiento con Merge Sort, cuya complejidad es O(k log k), donde k representa los pedidos que cumplen el filtro. Por eso, la complejidad temporal promedio de mi requerimiento es O(p + k log k).

Aunque construir el índice por país durante la carga requiere O(n) en promedio, este proceso se realiza previamente y no afecta el tiempo medido de la consulta. La carga completa tiene un costo O(n log n) debido al ordenamiento general. Considero que esta implementación es eficiente porque aprovecha las tablas hash para reducir los recorridos y solo ordena los pedidos que cumplen las condiciones.